

Technical Report

Question: hi

Technical Report

Topic: Handling Minimal, Ambiguous User Input (“hi”) in a Conversational AI Service

Prepared for: Product & Engineering Teams

Date: 2026-10-01

1. Executive Summary

The goal of this report is to outline a robust, Kubernetes-centric solution for gracefully handling

2. Research Findings

Category	Findings
Important Concepts	Greeting / Social Politeness – “hi” is a common, informal way to in
Requirements	Context Acquisition – System must request or infer additional context.
Benefits	User Engagement – Warm reply lowers barrier to further interaction.
Challenges	Ambiguity – Cannot determine user intent from a single greeting.
Suggested Next Steps	1. Acknowledge with a friendly greeting. 2. Prompt user for more inf

3. Technical Architecture

3.1 Overview

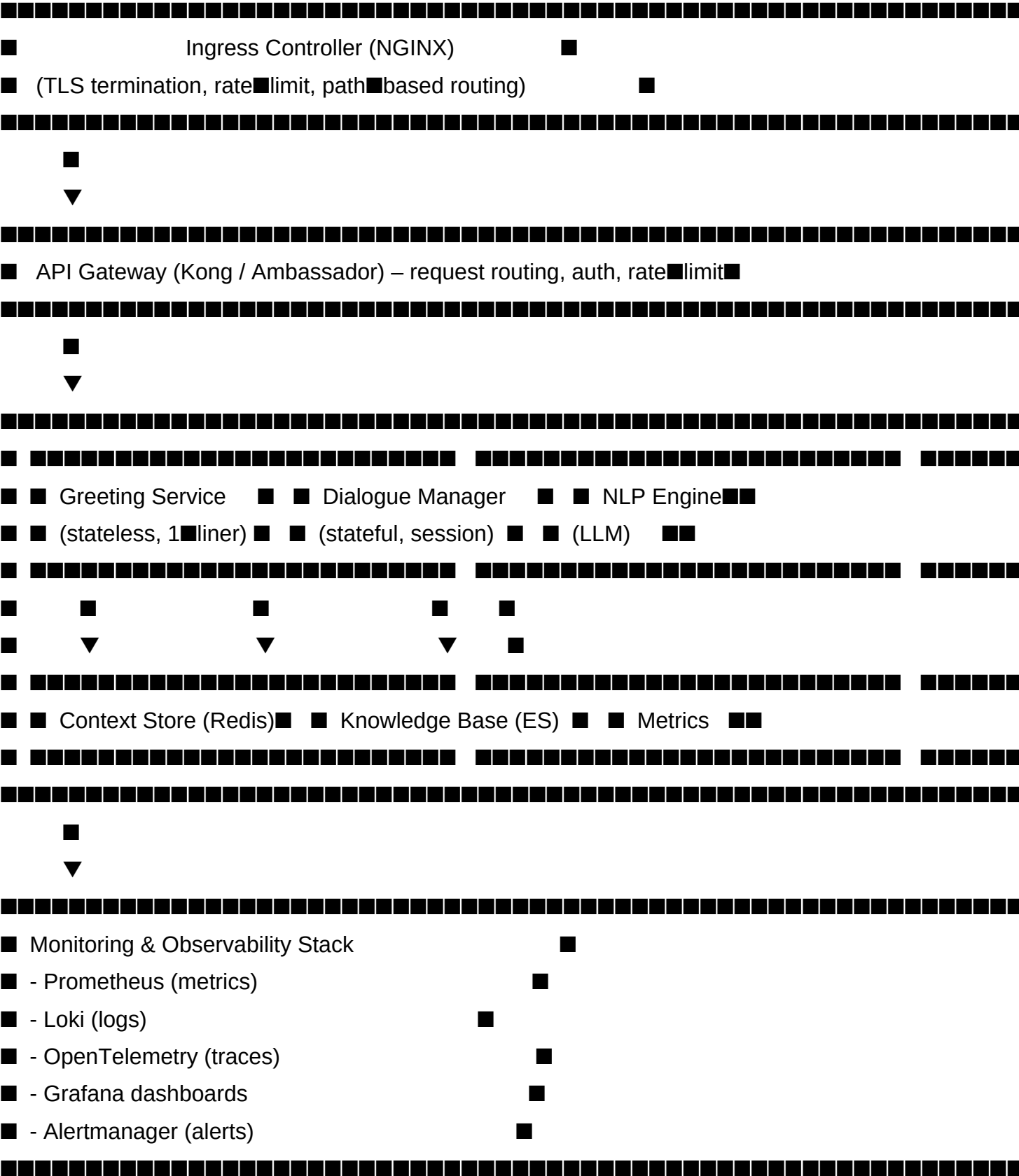
A Kubernetes-centric microservice stack delivers the conversational AI service. The architecture is

- Ingress & API Gateway – TLS termination, routing, auth, rate limiting.
- Greeting Service – Stateless, quick response to minimal inputs.
- Dialogue Manager – Stateful, session-aware context store.
- NLP Engine – LLM or rule-based engine for deeper understanding.

- 5. **Context Store** – Redis (or DynamoDB) for session persistence.
- 6. **Observability Stack** – Prometheus, Grafana, Loki, OpenTelemetry.

3.2 Textual Architecture Diagram

...



...

3.3 Kubernetes Components

Component	Purpose	Deployment Pattern
Deployments	Stateless services (Greeting, NLP) and stateful services (Dialogue Manager, Redis)	
StatefulSets	Persistent storage for Dialogue Manager (if local DB) and Redis.	PersistentVolume
Services	ClusterIP for internal load balancing. ClusterIP for internal, NodePort/LoadBalancer for external.	
Ingress	TLS termination, path routing, WAF.	NGINX Ingress Controller
ConfigMaps	Non-sensitive configuration (e.g., greeting templates).	Mounted as env vars or files
Secrets	Sensitive data (API keys, DB creds).	K8s Secrets + external vault integration
NetworkPolicies	Enforce pod-to-pod communication rules.	Only allow API Gateway → Services
HorizontalPodAutoscaler (HPA)	Scale services based on CPU/memory or custom metrics (e.g., requests per second).	
PodDisruptionBudget (PDB)	Ensure high availability during node maintenance.	Minimum 2 replicas

4. Implementation Steps

Step	Action	Tool / Resource	Notes
1	Define Service Contracts	OpenAPI / gRPC	Greeting → Dialogue Manager → NLP
2	Build Docker Images	Dockerfile, CI pipeline	Unit tests, linting
3	Create Helm Charts	Helm 3	Separate charts for greeting, dialogue, NLP, Redis
4	Set Up Ingress & API Gateway	NGINX Ingress, Kong	TLS certs, JWT auth
5	Deploy Context Store	Redis StatefulSet	Configure persistence, replication
6	Deploy Dialogue Manager	Deployment + HPA	Connect to Redis, expose metrics
7	Deploy Greeting Service	Deployment	Stateless, minimal latency
8	Deploy NLP Engine	Deployment	Containerized LLM or rule engine
9	Configure Observability	Prometheus Operator, Loki Operator, OpenTelemetry	Auto-discovery
10	Apply NetworkPolicies	NetworkPolicy YAML	Restrict traffic to necessary pods
11	Set Up CI/CD Pipeline	GitHub Actions / GitLab CI	Build → Test → Push → Helm chart update
12	Enable GitOps	ArgoCD / Flux	Automatic sync from Git repo
13	Implement Canary Releases	Flagger + Istio/Linkerd	Gradual traffic shift, monitor metrics
14	Define Alerts	Alertmanager	Latency, error rate, resource exhaustion
15	Run Smoke Tests	Post-deployment tests	Verify greeting, context acquisition, NLP flow
16	Promote to Production	ArgoCD promotion	After successful canary metrics

4.1 Handling the “hi” Input – Flow

1. **Client** → **Ingress** (TLS) → **API Gateway** (auth, rate limit).
2. **API Gateway** routes to **Greeting Service**.
3. **Greeting Service** returns a friendly message: “Hello! How can I help you today?” and creates a session ID.
4. Session ID is stored in **Context Store (Redis)** via **Dialogue Manager**.
5. If the user replies, the request includes the session ID; **Dialogue Manager** retrieves context.
6. **NLP Engine** processes the request, updates context, and returns a response.
7. All interactions are logged to **Loki** and metrics exposed to **Prometheus**.

5. Conclusion

By decomposing the conversational AI into dedicated microservices—each responsible for a single component—